High-Throughput Distributed Data Path

Project explained in simple terms — components, workflow and architecture

> **The project statement**
>
> "Tripled data-path throughput while holding p99 retrieval latency under 40 ms, as measured by load tests, by optimizing high-throughput storage and processing paths with distributed state, quorum replication, and synchronization patterns."

How to read this document: Section 1 gives the whole story in plain words. Section 2 explains the jargon. Section 3 walks through every component one by one. Section 4 follows a request through the system step by step. Section 5 shows the architecture diagram and explains every box, arrow and line style in it. Section 6 shows what was changed to get the 3x gain.

Note: The original statement is a one-line summary. The detailed design below is a typical, realistic way such a system is built, so specific numbers (like batch sizes or shard counts) are representative examples rather than exact figures from the original project.

Contents

1. The Project in Simple Terms 3
   1.1 The key numbers, explained 4
   1.2 Why holding p99 is the hard part 4
2. Key Words Made Simple 5
3. Components — Each One Explained 7
   A. Clients and Load-Test Generator 7
   B. Load Balancer 8
   C. Request Router 8
   D. Processing Workers (Stateless Pool) 9
   E. Hot-Data Cache 10
   F. Storage Cluster (Shards and Replicas) 11

G. Synchronization Patterns 12
H. Coordination Service 13
I. Observability and Load-Test Harness 14
4. Workflow — Following a Request End to End 15
4.1 Write path — saving data (PUT key = value) 15
4.2 Read path — fetching data (GET key) 16
4.3 Background workflows (always running quietly) 16
4.4 Failure workflows — what happens when things go wrong 17
4.5 Load-test workflow — how the result was proven 18
5. Architecture Diagram 19
5.1 How to read the diagram 21
5.2 Every box in the diagram 21
5.3 Every numbered arrow in the diagram 22
5.4 Line styles (legend at the bottom of the diagram) 23
5.5 The diagram as a story 24
6. What Was Changed to Reach 3x 25

1. The Project in Simple Terms

Imagine a very busy library. Thousands of people every second want to **drop off** books (writing data) and **pick up** books (reading data). The library was getting crowded: the queue was long and only a limited number of people could be served each second.

This project rebuilt how the library works so that it could serve **three times as many people per second** — and, at the same time, made sure that even the slowest pickups (the unlucky 1 in 100) still finished in **under 40 milliseconds** (40 thousandths of a second — faster than a blink, which takes about 100–150 ms).

It did this with three big ideas:

- **Distributed state** — instead of one giant shelf, the data is split across many machines ("shards"), so many machines work in parallel. Each machine owns a slice of the data.
- **Quorum replication** — every piece of data is copied to 3 machines. A write is only "done" once a majority (2 of 3) has safely saved it. A read asks a majority too. This way, data survives a machine crash and nobody waits for the slowest machine.
- **Synchronization patterns** — a set of rules (leaders, leases, version numbers, repair jobs) so that all those copies on different machines stay in agreement, even when machines are slow, crash or lose network connection.

On top of that, the **processing path** (the code that handles each request) was made more efficient: requests are grouped into batches, sent without waiting one-by-one (pipelining), and popular data is kept in a fast memory cache.

Finally, every claim was **proven with load tests**: a tool fired simulated traffic at the system, increased it step by step, and recorded throughput and the p99 latency at each step. The result: 3x the throughput of the old system, with p99 still below 40 ms.

> **The whole project in one sentence**
>
> Split the data across many machines, keep 3 copies of everything, finish each operation as soon as 2 copies agree, keep the copies in sync with clear rules, make each request do less waiting — and prove it with load tests.

1.1 The key numbers, explained

Term in the statement	What it means in plain words
Data path	The full route a piece of data travels: from the app that sends it, through the network and processing code, into storage — and back out again when someone reads it.
Throughput	How many operations (reads + writes) the system completes per second. Think “customers served per second”.
Tripled throughput	The system now completes about 3x as many operations per second as before, on comparable hardware (e.g. from ~50,000 to ~150,000 ops/sec — example values).
Latency	How long one single request takes from start to finish. Think “how long one customer waits”.
p99 latency	Sort all request times from fastest to slowest. p99 is the time at the 99th percentile: 99% of requests are faster than this, 1% are slower. It measures the “unlucky” users, not just the average one.
Under 40 ms	Even that unlucky 1% got their data back in less than 40 milliseconds.
Retrieval	Reading / fetching data (the “pickup” side).
Load tests	Controlled experiments that send realistic, heavy simulated traffic to measure what the system can really do.

1.2 Why holding p99 is the hard part

Making a system faster on average is relatively easy. Making it handle **3x more traffic** without the slowest requests getting slower is hard, because more traffic means more queues, more contention for disks and locks, and more chances that one slow machine delays a request. Averages hide those problems; p99 exposes them. That is why the project tracked p99 specifically, and why the design avoids ever waiting for the slowest machine (quorum = wait for the fastest majority).

2. Key Words Made Simple

Word	Simple meaning	Analogy
Node	One machine (server) in the cluster.	One librarian at one desk.
Cluster	A group of nodes working together as one system.	The whole library staff.
Shard / partition	A slice of the data owned by a specific group of nodes.	Shelves A–H, I–Q, R–Z.
Replica	A copy of a shard stored on another node.	A photocopy kept at another desk.
Leader	The one replica that decides the order of writes for a shard.	The head librarian of a section.
Follower	A replica that copies what the leader does.	Assistants who copy the head librarian's log.
Quorum	A majority of replicas (2 of 3).	"Two out of three witnesses signed it."
N, W, R	N = copies kept, W = copies that must confirm a write, R = copies asked on a read.	3 copies, 2 must sign, ask 2.
Consistent hashing	A math trick that maps every key to a shard so that adding a node moves only a little data.	A seating chart that barely changes when a new table is added.
WAL (write-ahead log)	A file where every change is appended first, so it survives a crash.	Writing in pen in a diary before doing anything.
Memtable	A sorted table in memory holding the newest writes.	Sticky notes on the desk.
SSTable	An immutable, sorted file on disk created when the memtable fills up.	Sticky notes filed into a bound folder.
Compaction	Background merging of SSTables to remove old versions and keep reads fast.	Re-organising folders at night.
Bloom filter	A tiny memory structure that quickly says "this key is definitely NOT in this file".	A sign on a folder listing what it doesn't contain.

Word	Simple meaning	Analogy
Lease	A time-limited permission (e.g. “you are leader for 10 s”) that must be renewed.	A parking ticket that expires.
Fencing token	An ever-increasing number given to each new leader so stale leaders are ignored.	A new key code issued each time the lock changes.
Version number	A counter attached to each value so newer data always wins.	Document v1, v2, v3.
Read repair	Fixing an outdated replica when a read notices it is behind.	Updating a stale photocopy when you spot it.
Anti-entropy	A background job that compares replicas and fixes differences.	A nightly stock-take between desks.
Merkle tree	A tree of hashes that lets two nodes find differences without sending all data.	Comparing chapter checksums instead of whole books.
Hinted handoff	If a replica is down, another node holds its writes and delivers them later.	A neighbour signing for your parcel.
Batching	Grouping many small operations into one bigger one.	Carrying 20 books in a trolley instead of one at a time.
Pipelining	Sending the next request without waiting for the previous answer.	Placing several orders at the counter in one go.
Backpressure	Slowing down senders when the system is near its limit, to avoid collapse.	A “please wait” sign at the door.
Idempotency key	A unique ID per request so a retry is never applied twice.	A receipt number that stops double charging.

3. Components — Each One Explained

The system has nine main parts, labelled **A to I** to match the architecture diagram in Section 5. Part **G (Synchronization Patterns)** is not a single box — it is a set of rules applied across the storage cluster and coordination service, so it is described here and shown in the diagram through the replication, anti-entropy and lease arrows.

Label	Component	One-line job
A	Clients & load-test generator	Send reads and writes into the system.
B	Load balancer	Spread incoming requests evenly.
C	Request router	Work out which shard owns each key; use the cache for reads.
D	Processing workers	Validate, batch and pipeline requests efficiently.
E	Hot-data cache	Answer popular reads from memory in under 1 ms.
F	Storage cluster (shards + replicas)	Durably store data as 3 copies per shard.
G	Synchronization patterns	Keep all copies correct and in agreement.
H	Coordination service	Track membership, leaders, leases and the partition map.
I	Observability & load-test harness	Measure throughput and p99; prove the results.

A. Clients and Load-Test Generator

What it is (simple): The programs that use the system. In production these are other apps and services that save and fetch data. During testing, a load-test generator pretends to be thousands of clients at once.

Everyday analogy: The customers walking into the library — and, during testing, a crowd of actors hired to behave like real customers.

How it works — step by step detail

- Each client uses a small **client library** that keeps a pool of open network connections, so it doesn't pay the cost of opening a new connection for every request.
- Every request carries a **key** (e.g. user-42), an **operation** (GET or PUT), an optional **value**, and an **idempotency key** (a unique request ID) so a retry never gets applied twice.

- Each request has a **timeout / deadline** (e.g. 40 ms budget for reads). If time runs out the client gives up or retries rather than waiting forever.
- Retries use **exponential backoff with jitter** (wait a bit, then a bit longer, with randomness) so failures don't create a stampede of retries.
- The load-test generator replays a realistic mix (for example 80% reads / 20% writes, with some keys far more popular than others) at a controlled rate.

How it helped the goal (3x throughput, p99 < 40 ms)

- Connection pooling removes repeated connection set-up cost — more requests per second for free.
- Deadlines stop slow requests from piling up and dragging down p99.
- Realistic test traffic made sure the 3x / 40 ms result reflects real use, not an easy benchmark.

B. Load Balancer

What it is (simple): The front door. It receives every request and passes it to one of several identical request-router instances.

Everyday analogy: A greeter at the entrance pointing each customer to the shortest queue.

How it works — step by step detail

- Keeps a list of healthy router instances; checks each one every few seconds with a **health check**.
- Uses a strategy such as **least connections** (send to the least busy router) rather than plain round-robin, so no single router gets overloaded.
- Removes unhealthy routers automatically and adds new ones when the system scales out.
- Can apply simple **rate limiting** at the edge to protect the system from sudden floods.

How it helped the goal (3x throughput, p99 < 40 ms)

- Even spreading of load keeps every router below its comfortable limit, which keeps queues short and p99 low.
- More routers can be added at any time, which is part of how throughput was multiplied.

C. Request Router

What it is (simple): The "brain" that decides where each request goes. It knows which shard owns each key and which replica is currently that shard's leader.

Everyday analogy: A receptionist with a seating chart: “Surnames A–H, go to desk 1; the head librarian there today is Alice.”

How it works — step by step detail

- **Consistent hashing:** the router hashes the key (turns it into a number) and places it on a ring. Each shard owns a section of the ring. This decides the shard in microseconds without asking anyone.
- **Virtual nodes:** each physical shard owns many small sections of the ring, so data spreads evenly and adding a machine only moves a small fraction of keys.
- **Partition map cache:** the router keeps a local copy of the map “shard → leader + followers”, received from the coordination service (H). It is updated automatically by a watch whenever leadership changes.
- **Reads:** first checks the hot-data cache (E). On a cache hit the answer returns immediately. On a miss, it forwards the read to a worker.
- **Writes:** always forwarded to a worker, which sends them to the shard leader.
- **Stale map handling:** if a node replies “I’m not the leader any more”, the router refreshes its map and retries once — no manual action needed.

How it helped the goal (3x throughput, p99 < 40 ms)

- Routing is a pure local calculation — no extra network hop to look up “who owns this key”.
- Even data spread means no single shard becomes a bottleneck, so adding shards adds throughput nearly linearly.

D. Processing Workers (Stateless Pool)

What it is (simple): A pool of identical worker processes that do the actual request handling: checking the request, grouping requests together, and talking to the storage cluster.

Everyday analogy: Library porters with trolleys. Instead of running to the shelves with one book at a time, they wait a split second, load up a trolley headed for the same section, and push it in one trip.

How it works — step by step detail

- **Stateless:** workers keep no permanent data themselves. Any worker can handle any request, so you can add or remove workers freely.
- **Validation:** checks the request is well-formed and within size limits.
- **Micro-batching:** requests going to the same shard are grouped. A batch is sent when it reaches a size limit (e.g. 64 operations) or a tiny time limit (e.g. 1–2 ms), whichever comes first — so batching never adds noticeable delay.

- **Pipelining:** the worker sends the next batch to a shard without waiting for the previous batch's answer. Many batches are "in flight" at once over the same connection.
- **Asynchronous (non-blocking) I/O:** a worker never sits idle waiting on the network; while one request waits, it processes others.
- **Efficient encoding:** data is sent in a compact binary format instead of verbose text, so less CPU is spent converting it and less data goes over the network.
- **Bounded queues + backpressure:** each worker's queue has a maximum size. If it fills up, the worker tells upstream to slow down instead of letting latency explode.
- **Request hedging for reads (optional):** if one replica is unusually slow to answer, a second copy of the read is sent to another replica and the first answer wins — this specifically cuts p99.

How it helped the goal (3x throughput, p99 < 40 ms)

- Batching + pipelining is one of the biggest sources of the 3x gain: far fewer network round trips and disk syncs per operation.
- Small batch time limits and bounded queues keep latency predictable, protecting the 40 ms p99.
- Stateless design means throughput grows simply by adding workers.

E. Hot-Data Cache

What it is (simple): A fast in-memory store that keeps copies of the most frequently read items, close to the router.

Everyday analogy: A small "most requested" shelf right next to the front desk, so popular books don't need a trip to the back stacks.

How it works — step by step detail

- On a read, the router checks the cache first. A **hit** returns in under 1 ms.
- On a **miss**, the value is fetched from storage and then placed in the cache for next time.
- Least-recently-used items are evicted (thrown out) when the cache is full.
- **Invalidation on write:** when a key is written, its cache entry is removed or updated (using its version number), so readers never see old data for long.
- Each entry also has a short **time-to-live (TTL)** as a safety net.
- **Request coalescing:** if 1,000 clients ask for the same missing key at once, only one fetch goes to storage; the others wait for that single answer (prevents a "thundering herd").

How it helped the goal (3x throughput, p99 < 40 ms)

- A large share of reads never touch the storage cluster, freeing storage capacity for more traffic.

- Popular (“hot”) keys no longer overload one shard — a major cause of bad p99 tail latency.

F. Storage Cluster (Shards and Replicas)

What it is (simple): Where the data actually lives. The data is split into shards, and each shard is stored as 3 replicas on 3 different machines: one leader and two followers.

Everyday analogy: Several sections of the library (A–H, I–Q, R–Z). Each section has one head librarian and two assistants who keep identical copies of every record.

How it works — step by step detail

- **Sharding (distributed state):** each shard holds only its slice of keys, so all shards work in parallel. Doubling shards roughly doubles capacity.
- **Replication factor N = 3:** every write is stored on 3 machines, ideally in different racks or availability zones, so one failure never loses data.
- **Leader:** receives all writes for its shard, gives each write a sequence/version number, and sends it to the followers.
- **Followers:** append the leader’s writes in the same order and can serve reads.

Inside each storage node (how one machine saves data fast)

- **1. WAL (write-ahead log):** the write is first appended to the end of a log file. Appending is the fastest possible disk operation. Many writes are flushed to disk together in one “group commit”, instead of one disk sync per write.
- **2. Memtable:** the write is then placed in a sorted in-memory table, so recent data can be read instantly.
- **3. SSTables:** when the memtable is full (e.g. 64 MB), it is written out as a sorted, never-changed file on disk, and a new memtable starts.
- **4. Bloom filters + index:** each SSTable has a small bloom filter and index in memory, so a read can skip files that cannot contain the key and jump straight to the right spot in files that might.
- **5. Compaction:** in the background, SSTables are merged, old versions and deleted items are thrown away, and the number of files a read must check stays small. Compaction speed is limited so it never steals too much disk bandwidth from live requests (which would hurt p99).
- **6. Block cache:** recently read disk blocks are kept in memory on each node.

Quorum replication — the math in plain words

- **N = 3** copies, **W = 2** copies must confirm a write, **R = 2** copies are consulted on a read.
- Because **W + R = 4 > N = 3**, the 2 copies you read always overlap with at least 1 of the 2 copies that confirmed the latest write. So a quorum read always sees the latest confirmed write.

- The leader writes locally and to both followers at the same time, but replies "success" as soon as **2 of 3** have it — it never waits for the slowest copy. The third copy catches up shortly after.
- One machine per shard can be down or slow and the shard keeps working for both reads and writes.

How it helped the goal (3x throughput, p99 < 40 ms)

- Sharding is the main way throughput was multiplied: more shards = more parallel work.
- Waiting for 2 of 3 (not 3 of 3) removes the slowest machine from the critical path — the single most important trick for keeping p99 under 40 ms.
- Group commit in the WAL and the LSM design (WAL → memtable → SSTable) make writes cheap; bloom filters and caches make reads cheap.

G. Synchronization Patterns

What it is (simple): The rules that keep many copies of data on many machines correct and in agreement, even when machines crash, run slowly or get cut off from the network.

Everyday analogy: The library's rulebook: who is in charge of each section, how long they stay in charge, how to number each record, and how assistants check and fix their copies.

How it works — step by step detail

- **Leader election:** for each shard, exactly one replica is chosen as leader through the coordination service. Only the leader orders writes, so there are never two conflicting orders.
- **Leases:** a leader holds its role for a fixed time (e.g. 10 seconds) and must keep renewing it. If it crashes, the lease expires and a new leader is elected automatically.
- **Fencing tokens / epochs:** each new leader gets a higher number. Followers reject messages carrying an older number, so a "zombie" old leader that wakes up can't overwrite newer data.
- **Per-key version numbers:** every value is stored with a version. When copies disagree, the higher version wins. Writes can use **compare-and-set** ("only update if the version is still 7") to avoid lost updates.
- **Ordered log replication:** followers apply writes in exactly the leader's order, using sequence numbers to detect and request anything missing.
- **Read repair:** if a quorum read sees that one replica has an older version, it sends the newer value to that replica in the background.
- **Anti-entropy with Merkle trees:** a periodic background job compares replicas using trees of hashes. Only the parts that differ are sent — cheap even for huge datasets.
- **Hinted handoff:** if a replica is temporarily down, a neighbouring node stores a "hint" (the write it missed) and delivers it when the replica comes back.

- **Idempotent writes:** writes carry request IDs, so a retried write is recognised and applied only once.
- **Lock-free / fine-grained concurrency inside a node:** instead of one big lock, each node uses per-partition queues or fine-grained locks, so threads don't queue behind each other.

How it helped the goal (3x throughput, p99 < 40 ms)

- Correctness is maintained without a slow "stop everything and agree" step for every operation, so the fast path stays fast.
- Read repair, anti-entropy and hinted handoff happen in the background — they fix data without adding latency to user requests.
- Replacing coarse locks with fine-grained or lock-free structures removed contention that used to cap throughput and inflate p99.

H. Coordination Service

What it is (simple): A small, highly reliable cluster (for example etcd or ZooKeeper) that stores the critical "who is who" information for the whole system.

Everyday analogy: The library manager's office with the official staff roster and seating chart. Everyone checks with it when something changes, but customers never go there.

How it works — step by step detail

- **Membership:** keeps a list of live storage nodes using heartbeats; a node that stops sending heartbeats is marked down.
- **Leader election & leases:** grants and renews shard leadership (see G).
- **Partition map:** stores the official mapping "shard → leader + followers". Routers watch it and get notified instantly when it changes.
- **Rebalancing:** when nodes are added or removed, it coordinates moving shards so data stays evenly spread.
- It runs its own consensus protocol (Raft or similar) across 3 or 5 machines, so it survives failures itself.
- It is **not** on the normal request path — routers cache the map locally, so it adds zero latency to everyday reads and writes.

How it helped the goal (3x throughput, p99 < 40 ms)

- Keeping coordination off the hot path means it never becomes a throughput bottleneck.
- Fast, automatic failover keeps latency stable during failures instead of causing long stalls.

I. Observability and Load-Test Harness

What it is (simple): The measuring tools: metrics, latency histograms, dashboards, alerts, and the load-testing setup that proved the result.

Everyday analogy: Stopwatches and counters at every desk, plus a rehearsal day where the library is deliberately flooded with actors to see how it copes.

How it works — step by step detail

- Every component reports **throughput** (operations per second) and **latency histograms** (how many requests took 0–1 ms, 1–2 ms, …), from which p50, p95, p99 are calculated.
- Detailed metrics per stage (queue time, network time, disk time, replication time) show **where** the time goes, which guided each optimization.
- **Load-test method:** (1) warm up the system, (2) ramp traffic up in steps, (3) hold each step for several minutes, (4) record throughput and p99 at each step, (5) find the highest throughput at which p99 stays under 40 ms.
- The same test is run on the **old** design (baseline) and the **new** design, with the same hardware and traffic mix, so the 3x comparison is fair.
- Tests include **failure scenarios** (kill a node, slow a disk) to confirm p99 holds even during problems.
- The test generator measures latency using the **intended send time** (avoiding "coordinated omission"), so slow periods aren't hidden from the numbers.
- Dashboards and alerts watch p99 in production after rollout.

How it helped the goal (3x throughput, p99 < 40 ms)

- Turned the claim "3x faster with p99 < 40 ms" into evidence rather than opinion.
- Pinpointed the real bottlenecks (round trips, lock contention, compaction, hot keys) so effort went where it mattered.

4. Workflow — Following a Request End to End

This section follows a request through the system step by step. The numbers in brackets like **[1]** match the numbered circles on the architecture diagram in Section 5.

4.1 Write path — saving data (PUT key = value)

1. **Client sends the write [1].** The app calls PUT("user-42", {…}) with a unique request ID and a deadline. The request goes over an already-open pooled connection to the load balancer.
2. **Load balancer picks a router [2].** It chooses the healthiest, least-busy router instance and forwards the request.
3. **Router finds the shard.** It hashes "user-42" on the consistent-hash ring → Shard 1. From its locally cached partition map [8], it knows Shard 1's leader is on Node A.
4. **Router invalidates the cache entry.** It marks "user-42" as stale in the hot-data cache, so no one reads an old value afterwards.
5. **Router hands the write to a worker [4].** The worker validates the request (size, format).
6. **Worker batches it.** The write joins a small batch of other writes heading to Shard 1. The batch is sent when it is full or after ~1–2 ms, whichever comes first.
7. **Worker sends the batch to the Shard 1 leader [5].** Thanks to pipelining, it doesn't wait for earlier batches to finish.
8. **Leader checks its lease and orders the write.** It confirms it is still the valid leader (current fencing token), assigns the next sequence number and a new version number to the key, and checks the request ID to drop duplicates.
9. **Leader saves locally.** The batch is appended to its WAL (one group disk-sync for the whole batch) and inserted into its memtable.
10. **Leader replicates in parallel [6].** At the same time, it sends the batch to Follower 1 and Follower 2.
11. **Followers save.** Each follower checks the sequence number (nothing missing, correct leader epoch), appends to its own WAL and memtable, and replies "saved".
12. **Quorum reached.** As soon as 2 of 3 copies (the leader + the fastest follower) have it, the write is committed. The leader does not wait for the third copy.
13. **Acknowledgement flows back [7].** Leader → worker → router → load balancer → client: "success, version 18".
14. **The slow follower catches up.** The third follower finishes shortly after. If it was down, hinted handoff or anti-entropy fills the gap later.
15. **Metrics recorded [10].** Each stage reports its timing to observability.

4.2 Read path — fetching data (GET key)

1. **Client sends the read [1]** with a deadline (well within the 40 ms budget).
2. **Load balancer forwards it [2]** to a router.
3. **Router checks the hot-data cache [3].** If the key is there and current (a **cache hit**), the value is returned immediately — usually under 1 ms. Done.
4. **Cache miss:** the router hashes the key to find its shard and the replicas from its partition map, then hands the read to a worker [4].
5. **Worker sends a quorum read [5].** It asks 2 of the 3 replicas (usually the leader plus the nearest/fastest follower), batched with other reads to the same shard.
6. **Replicas look up the key.** Each checks memtable first, then uses bloom filters to skip SSTables that can't contain the key, and reads only the right block (often from block cache).
7. **Replicas reply** with the value and its version number.
8. **Worker compares versions.** The highest version wins. Because W + R > N, at least one of the two replies is guaranteed to be up to date.
9. **Hedging (if needed).** If one replica is unusually slow (e.g. above its normal p95), the worker sends the read to the third replica and uses whichever answers first.
10. **Read repair.** If one replica returned an older version, the worker sends it the newer value in the background — the client doesn't wait for this.
11. **Value returned to the client [7]**, and placed into the hot-data cache for next time.
12. **Metrics recorded [10]** — this read's latency joins the histogram that produces the p99.

4.3 Background workflows (always running quietly)

Background job	What it does	Why the user never notices
Heartbeats & lease renewal [9]	Leaders renew their leases with the coordination service every few seconds; nodes send heartbeats.	Tiny messages, off the request path.
Partition-map watch [8]	Routers receive updates the moment leadership or shard ownership changes.	Router keeps serving from its cached map meanwhile.
Memtable flush	Full memtables are written to disk as SSTables.	New writes go to a fresh memtable at once.
Compaction	Merges SSTables, removes old versions.	Speed-limited so it never starves live traffic.

Background job	What it does	Why the user never notices
Anti-entropy (dashed arrows)	Replicas compare Merkle trees and fix differences.	Runs at low priority, sends only differences.
Hinted handoff	Delivers writes missed by a node that was briefly down.	Happens after the node returns.

4.4 Failure workflows — what happens when things go wrong

A leader crashes

1. The leader stops renewing its lease and stops sending heartbeats.
2. After the lease expires (a few seconds), the coordination service elects the most up-to-date follower as the new leader and gives it a higher fencing token.
3. The partition map is updated; routers are notified by their watch [8].
4. Requests that were in flight get a “not leader” or timeout reply and are retried once to the new leader, using the same request ID so nothing is applied twice.
5. If the old leader comes back, its old fencing token is rejected — it rejoins as a follower and catches up.

A follower is slow or down

- Writes still succeed because leader + the other follower = 2 of 3 (quorum).
- Reads use the two healthy replicas; hedging avoids waiting on the slow one.
- Hinted handoff and anti-entropy bring the follower up to date when it recovers.

Network partition (a group of machines can’t talk to the others)

- Only the side that holds a majority can elect a leader and accept writes. The minority side refuses writes, so the data can never split into two conflicting versions.

A key becomes extremely popular (hot key)

- The hot-data cache absorbs most of its reads; request coalescing ensures only one fetch hits storage on a miss.
- Reads for that key can be spread across all three replicas.

Traffic spike beyond capacity

- Bounded queues fill, backpressure tells upstream to slow down, and excess requests are rejected quickly with a “try again” instead of making every request slow. This protects p99 for the requests that are accepted.

4.5 Load-test workflow — how the result was proven

1. **Define the target:** maximum throughput while p99 read latency stays under 40 ms.
2. **Build a realistic workload:** realistic read/write ratio, value sizes, and key popularity (a few very popular keys, many rarely used ones).
3. **Measure the baseline:** run the test against the old design and find its maximum throughput under the 40 ms p99 limit.
4. **Warm up** the new system so caches and connections are in a normal state.
5. **Ramp traffic in steps** (e.g. +10% every few minutes), holding each step long enough to get stable numbers.
6. **Record at each step:** throughput, p50, p95, p99, error rate, CPU, disk and network use.
7. **Find the knee:** the highest step where p99 is still under 40 ms. Compare it to the baseline — the new design reached about 3x.
8. **Repeat with failures** (kill a node, slow a disk) to confirm the p99 limit holds during incidents.
9. **Repeat several runs** to make sure the result is consistent, not a lucky one-off.

5. Architecture Diagram

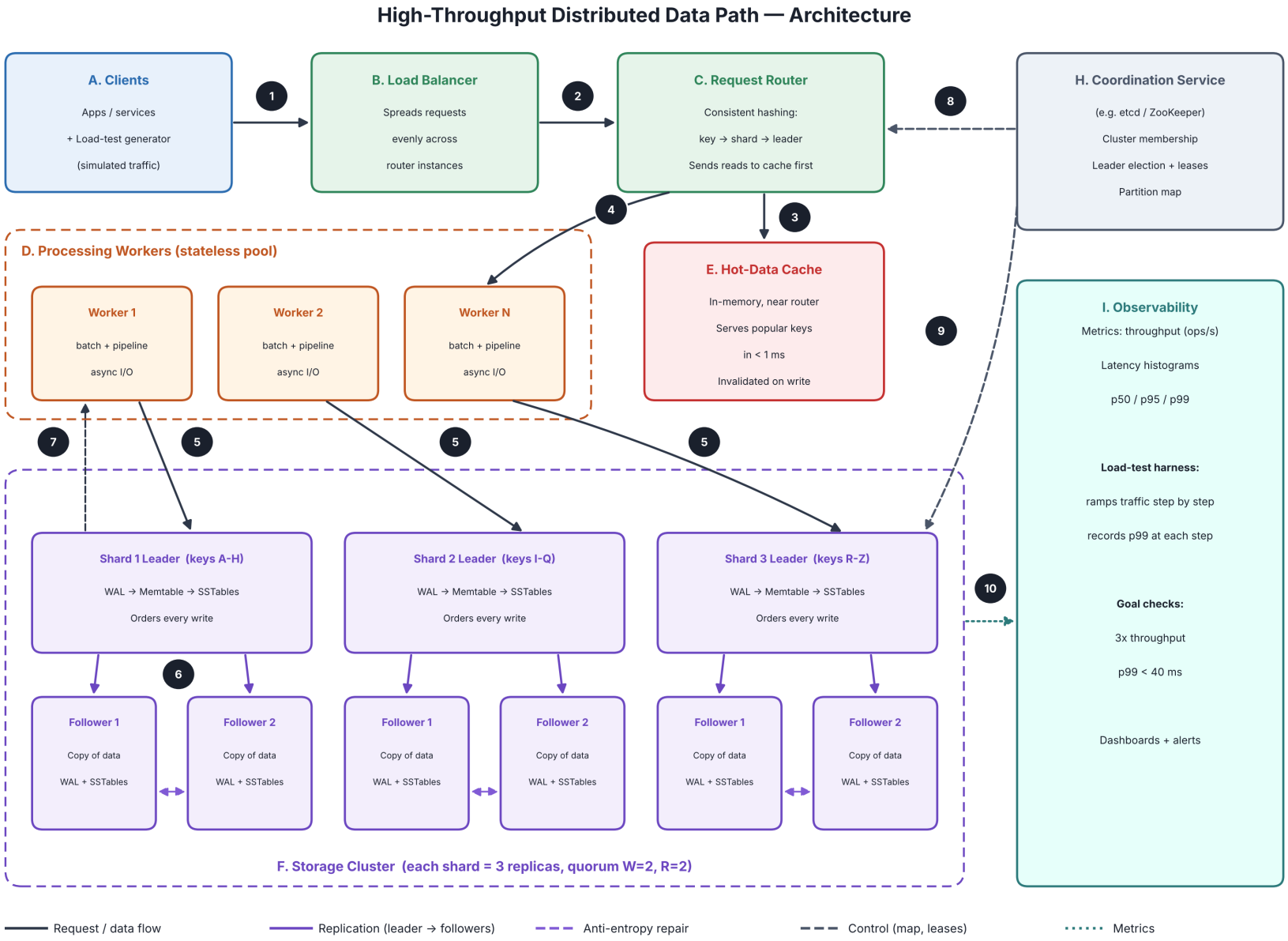


Figure 1 — End-to-end architecture. Letters A–I label components; numbered circles 1–10 label the flow steps.

5.1 How to read the diagram

- **Top row** — how requests enter: Clients (A) → Load Balancer (B) → Request Router (C). The Coordination Service (H) sits to the right, feeding the router its map.
- **Middle row** — fast processing: the pool of Processing Workers (D) and the Hot-Data Cache (E).
- **Bottom row** — durable storage: the Storage Cluster (F) with 3 shards; each shard has one leader and two followers.
- **Right column** — Observability (I), which collects metrics from everything and runs the load tests.
- **Synchronization patterns (G)** are not a separate box: they are the replication arrows, the anti-entropy arrows between followers, and the lease/control arrows from the Coordination Service.
- **Colours** group parts by role: blue = clients, green = entry/routing, orange = processing, red = cache, purple = storage, grey = coordination, teal = observability.

5.2 Every box in the diagram

Box	What is written inside	What it means
A. Clients	Apps / services; + Load-test generator; (simulated traffic)	Real applications in production, and a traffic generator during testing.
B. Load Balancer	Spreads requests evenly across router instances	Picks the least-busy healthy router for each request.
C. Request Router	Consistent hashing: key → shard → leader; Sends reads to cache first	Calculates which shard owns a key and who its leader is; checks the cache before going to storage.
D. Processing Workers (dashed group)	Worker 1, Worker 2, Worker N — batch + pipeline, async I/O	Identical, stateless workers. “N” means as many as needed. Each batches requests and keeps many in flight.
E. Hot-Data Cache	In-memory, near router; Serves popular keys in < 1 ms; Invalidated on write	Fast memory copy of popular data, cleared when that data changes.
F. Storage Cluster (dashed group)	Each shard = 3 replicas, quorum W=2, R=2	All data, split into shards; 3 copies each; 2 must agree on writes and reads.

Box	What is written inside	What it means
Shard 1/2/3 Leader (keys A-H, I-Q, R-Z)	WAL → Memtable → SSTables; Orders every write	The leader of each slice of keys. Saves writes via log → memory → disk files, and decides the order of writes.
Follower 1 / Follower 2	Copy of data; WAL + SSTables	The two extra copies of each shard. They copy the leader and can serve reads.
H. Coordination Service	(e.g. etcd / ZooKeeper); Cluster membership; Leader election + leases; Partition map	The reliable source of truth for who is alive, who leads each shard, and which shard owns which keys.
I. Observability	Metrics, latency histograms, p50/p95/p99; Load-test harness; Goal checks: 3x throughput, p99 < 40 ms; Dashboards + alerts	Measures everything and proves the project goals were met.

5.3 Every numbered arrow in the diagram

#	From → To	What travels along it	Simple explanation
1	Clients → Load Balancer	Read and write requests	Customers walk in the front door.
2	Load Balancer → Request Router	The same request, to a chosen router	Greeter sends each customer to a free receptionist.
3	Request Router → Hot-Data Cache	Read lookups; cache invalidations on writes	Check the “popular shelf” first; remove outdated copies on writes.
4	Request Router → Processing Workers	Writes and cache-miss reads	Hand the job to a porter.
5 (x3)	Workers → Shard 1, 2, 3 leaders	Batched, pipelined writes and quorum reads	Porters push full trolleys to the right section — all sections in parallel.

#	From → To	What travels along it	Simple explanation
6	Leader → Follower 1 and Follower 2 (solid purple)	Replicated writes in order	Head librarian sends copies of every new record to both assistants at once.
7	Leader → Workers (dashed, upward)	Acknowledgement once 2 of 3 copies are saved; read results	“Done!” goes back as soon as two copies are safe — then back to the client.
8	Coordination Service → Request Router (dashed grey)	Partition-map updates	Updated seating chart whenever a leader changes.
9	Coordination Service → Shard leaders (dashed grey)	Leader election, lease grants/renewals, fencing tokens	Manager confirms who is in charge, for how long.
10	Storage Cluster → Observability (dotted teal)	Metrics: throughput, latency, errors (all components report)	Stopwatch readings sent to the scoreboard.

5.4 Line styles (legend at the bottom of the diagram)

Line style	Meaning
Solid dark line	Request / data flow — the main path user requests take (arrows 1, 2, 3, 4, 5).
Solid purple line	Replication — leader sending writes to its followers (arrow 6).
Dashed purple double-arrow (between followers)	Anti-entropy repair — background comparison and fixing of replicas using Merkle trees.
Dashed grey line	Control traffic — partition map and leases from the coordination service (arrows 8, 9). Not on the user request path.
Dashed dark upward line	Acknowledgement / response going back (arrow 7).
Dotted teal line	Metrics flowing into observability (arrow 10).

5.5 The diagram as a story

A request enters at the top left (1), is passed to a router (2), which either answers from the cache (3) or hands the job to a worker (4). The worker groups it with similar requests and sends the batch to the right shard leader (5). The leader saves it and copies it to its followers (6); as soon as two copies are safe, the answer goes back up (7). Meanwhile, the coordination service keeps the router's map current (8) and keeps exactly one valid leader per shard (9), followers quietly repair any differences between themselves (dashed purple), and every part reports its numbers to observability (10), where load tests confirm 3x throughput with p99 under 40 ms.

6. What Was Changed to Reach 3x

The 3x gain did not come from one trick. It was the combined effect of removing waiting at every layer. The table below lists the typical changes and why each one matters.

Area	Before (typical bottleneck)	After (optimization)	Effect
Data layout	Data on one or few large nodes	Sharded across many nodes with consistent hashing	Parallel work; capacity grows by adding shards
Replication	Wait for all copies (3 of 3)	Quorum: wait for 2 of 3	Slowest machine removed from the critical path → lower p99
Network	One request = one round trip	Micro-batching + pipelining + connection pooling	Many fewer round trips per operation
Disk writes	One disk sync per write	WAL group commit	Many writes share one disk sync
Reads	Scan many files	Bloom filters, block cache, controlled compaction	Fewer disk reads per lookup
Popular data	Hot keys overload one shard	Hot-data cache + request coalescing	Storage protected; stable tail latency
Concurrency	Coarse locks, threads waiting	Fine-grained / lock-free structures, async I/O	CPU busy doing work instead of waiting
Coordination	Lookups on the request path	Cached partition map, watches, leases	Zero coordination cost per request
Overload	Unbounded queues → latency explosion	Bounded queues + backpressure + deadlines	Predictable p99 under heavy load
Slow replicas	Wait for a slow node	Hedged reads	Cuts the worst-case tail

Final takeaway

Throughput went up because work was split into parallel shards and each request did far fewer round trips, disk syncs and lock waits.

p99 stayed under 40 ms because the system never waits for the slowest copy (quorum), keeps popular data in memory, limits queues, and does repairs and housekeeping in the background.

Correctness was kept because synchronization rules — leaders, leases, fencing tokens, version numbers, read repair and anti-entropy — keep every copy in agreement.

The result is trustworthy because it was measured with realistic, repeatable load tests against the old system.